Week 5 Lecture Notes: Data Engineering for Noisy Sensor Data

Missing data, outliers, normalisation and leakage-free splits on a rover log

AI for Aerospace & Mechanical Engineering (PhD), Block 0

> **Provenance:** partly expanded from instructor notes. Section 1 expands slides 2 to 8 of the instructor's deck "Machine Learning and Decision Trees" (AI ⊃ ML ⊃ DL; feature extraction; classical methods cannot run on raw CSV files; the classification, clustering and regression taxonomy). Everything else is new content. Missing-data mechanisms follow R. J. A. Little and D. B. Rubin, *Statistical Analysis with Missing Data* (3rd ed., 2019), Ch. 1; cleaning, imputation, outlier rules, scaling and leakage are rewritten (not copied) from W3Schools ("Pandas - Cleaning Data", "ML Scaling", "ML Train/Test") and GeeksforGeeks ("Handling Missing Values in Machine Learning", "Detect and Remove the Outliers using Python", "Data Leakage"). LiDAR error types follow S. Thrun, W. Burgard, D. Fox, *Probabilistic Robotics* (2005), Sec. 6.3; the z-score bound is P. A. Samuelson, JASA 63 (1968); the Hampel filter follows R. K. Pearson, IEEE Trans. Control Syst. Technol. 10 (2002); blocked validation follows D. R. Roberts et al., *Ecography* 40 (2017).

1. The problem

The project rover (module M3) must brake before a wall. Its controller needs the LiDAR range **one second ahead**, $r(t + 1\,\mathrm{s})$, from the current range r, wheel speed v, longitudinal acceleration a and yaw rate ω. We have 16 test drives logged at 10 Hz in `data/sample/week05_rover_log.csv` (2533 rows). The naive route is: read the CSV, shift the range by 10 rows, drop NaN rows, split rows at random, fit. A linear model then misses by **2.73 m**, for a stop distance of one metre. Nothing is wrong with the model. The log contains every defect a real rover produces, and the naive route misreads each one:

489 leading LiDAR zeros (no echo, wall beyond 12 m) are read as "wall touching the sensor"; 41 random dropouts and 11 dust echoes pass as valid ranges; 19 NaN wheel speeds silently drop rows; 11 accelerometer spikes up to ±25 m/s² look like manoeuvres; two 3-s frozen-gyro segments look like a perfect turn rate; 25 re-sent rows become extra training data; and `shift(-10)` over concatenated drives takes the last ten targets of each run from the next drive.

The instructor's deck makes the general point: classical methods (decision trees, SVMs, naive Bayes, logistic regression) cannot run on a raw CSV, they need prepared features, and even deep networks, which learn their own features, need data that means what it says. This week is supervised **regression**, with the label taken from the log itself.

2. Audit before you clean

`audit_frame(frame, sentinels={"lidar_range_m": 0.0})` counts per column the NaN, ±inf and sentinel cells, the longest NaN run and the longest run of **identical** values: with 0.3 °/s gyro noise, 32 equal readings cannot be physics, though 51 wheel speeds of 0.0 just mean the rover is parked.

W3Schools lists four kinds of bad data (empty cells, wrong format, wrong data, duplicates); sensor logs add two more, **sentinels** and **frozen values**.

3. Missing data: the mechanism decides

Little and Rubin classify **why** a value is missing:

- **MCAR** (completely at random): unrelated to anything. A lost CAN frame.
- **MAR** (at random): depends on another *observed* column. GPS dropouts while a logged `in_hangar` flag is true.
- **MNAR** (not at random): depends on the missing value itself. The LiDAR is silent exactly when the wall is far.

Worked Example 1: mean imputation under MNAR

True ranges along a drive are 2, 4, 6, 8, 10, 13, 15, 17 m and the LiDAR maximum is 12 m. The true mean is $75/8 = 9.375$ m. Only the first five are observed, with mean $30/5 = 6.0$ m. Filling the three gaps with 6.0 m gives errors of -7, -9 and -11 m, and the mean after imputation is still 6.0 m: a bias of -3.4 m that no amount of data removes. On the rover log the true mean is 6.86 m and the observed one about 5.1 m. The honest information is "farther than 12 m": keep it as an indicator column, or restrict the model's domain to the sensor range, which is what the class recipe does.

The remedies: drop the row (MCAR, few rows lost); interpolate in time (short interior gaps, here at most 5 samples); median imputation plus a `_was_missing` indicator (weak features, MAR or MNAR); restrict the domain (MNAR caused by a sensor limit). Never interpolate across a long gap or across two runs. `MarkInvalid` first turns the sentinel and any value outside $[0.3, 12]$ m into NaN, so every remedy sees "unknown", not "0 m".

Worked Example 2: interpolating a gap

With $r(0) = 8.0$ m and $r(0.3) = 7.4$ m and the two samples between them missing,

$$r(t) = 8.0 + (7.4 - 8.0)\,\frac{t}{0.3}, \qquad r(0.1) = 7.8\ \text{m}, \quad r(0.2) = 7.6\ \text{m}.$$

`FillGaps(max_gap=5, time_column="time_s")` reproduces both and leaves longer gaps as NaN. At 10 Hz and under 2.2 m/s, a 0.5 s gap costs a few centimetres unless the rover brakes inside it.

4. Outliers: global versus local rules

The z-score rule flags $|x - \bar{x}|/\sigma > 3$. On small samples it cannot work: with the population standard deviation, **every** point satisfies (Samuelson, 1968)

$$|z_i| \le \sqrt{n-1},$$

so for $n \le 9$ no point can ever exceed 3. With the sample standard deviation the bound is $(n-1)/\sqrt{n}$ (Shiffler, 1988). The outlier also inflates the very σ that should expose it: this is **masking**.

Worked Example 3: one spike in eight samples

Seven accelerometer readings near zero and one spike of 25 m/s² give $\bar{x} = 3.16$, $\sigma = 8.26$ and $z = 2.645 < 3$: not flagged, and at the bound $\sqrt{7} = 2.6458$. The robust alternative replaces mean and

std by median and MAD (median absolute deviation): median 0.05, MAD 0.15, robust score $(25 - 0.05)/(1.4826 \cdot 0.15) \approx 112$. Flagged.

Robust and **global** is still not enough. Tukey's fences $[Q_1 - kR, Q_3 + kR]$ with $R = Q_3 - Q_1$ are robust, but computed over the whole log they flag 250 accelerometer samples, 239 of them genuine braking. The **Hampel filter** compares each sample with the median and MAD of its own 7-sample window, flags $|x_i - \tilde{x}| > 3 \cdot 1.4826 \cdot \mathrm{MAD}$ and replaces it with the window median: 117 flags, all near spikes or dust echoes. A dust echo of 1.2 m with the wall at 8 m is a valid range on its own; only its neighbours show it is wrong. Report the side effect too: the filter also nudges noisy samples by about 4 cm (range) and 0.3 m/s² (accel).

In-class Problem 1 repeats this on a dust echo in seven LiDAR readings, where $z = -\sqrt{6}$ is again the largest possible value and the robust score is 229.

5. Respect the run boundaries

The log is 16 separate drives concatenated. Interpolation, the Hampel window, rolling means and the target shift all look at neighbours and must stop at the end of a run; otherwise the last ten targets of run 3 are ranges from run 4. `PerGroup(stage, group_column="run_id")` is a **Composite**: it is a stage and holds a stage, applying it to each run separately. It accepts only stateless stages (a scaler per run would be a different model) and round-trips through the Week 3 JSON config. The class recipe `rover_cleaning_pipeline()`:

`SortAndDeduplicate` (2533 to 2508 rows), `MarkInvalid` on the range (530 cells to NaN), `PerGroup(HampelFilter)` on range and accel (167 cells changed), `PerGroup(FillGaps)` on range and speed (58 cells filled), `PerGroup(ForecastTarget)` (`range_ahead_m`, 10 rows later) and `DropNonFinite` (1859 rows with every feature and the target).

Nothing in this recipe is learned from data, so it is safe to run before the split. Most of the 649 dropped rows are the MNAR start of each run: the model's domain is "wall within sensor range". The factory loader `rover_range_forecast` (Week 4 Factory) runs the recipe and returns a `Dataset` whose `groups` field holds the run id.

6. Scaling, and the one rule

The z-score $(x - \mu)/\sigma$ suffers when outliers inflate σ; min-max $(x - \min)/(\max - \min)$ lets one outlier squeeze the rest; robust scaling $(x - \mathrm{median})/\mathrm{IQR}$ fails only on a constant column.

Worked Example 4: who is the nearest neighbour?

A query in steady driving $q = (5.0\ \mathrm{m}, 0.0\ \mathrm{m/s^2})$, a braking sample $A = (5.1, -1.0)$ and a steady sample $B = (6.2, 0.0)$. Raw distances: $d_A = 1.005$, $d_B = 1.200$, so k-NN copies the braking sample. Divide by the training stds (3.0 m and 0.4 m/s²): $d_A = \sqrt{(0.1/3)^2 + (1/0.4)^2} = 2.500$, $d_B = 1.2/3 = 0.400$. Distance models and gradient descent need scaling; decision trees (Week 7) do not.

The rule: every imputer, clipper and scaler *learns a number*. Learn it from the training rows only and apply the same number to the test rows. `ColumnStatStage` enforces this as a template method: `fit` calls `learn()` and stores `stats_`, `transform` calls `apply()` and raises `NotFittedError` if

`fit` never ran. A test value outside $[0, 1]$ after min-max scaling is a finding (the test drive went somewhere training did not), not a bug.

7. Leakage and honest splits

Leakage means the test set is not really unseen (GeeksforGeeks lists the causes: wrong split, preprocessing before the split, features unavailable at prediction time, pipeline errors). For time series the worst is the **random row split**: at 10 Hz each test row has training twins 0.1 s away. With 25 % test rows, the chance that a test row has at least one training neighbour is $1 - 0.25^2 = 0.9375$. Same k-NN model, four splits:

Split	k-NN RMSE [m]	Linear RMSE [m]	`check_split`
Random rows	0.319	0.098	LEAKY
Chronological	0.634	0.120	LEAKY (one run cut in two, targets overlap)
Chronological, gap = 10 rows	0.634	0.120	OK
By run (`GroupHoldoutSplit`)	**0.641**	**0.121**	OK

The random split halves k-NN's error because k-NN memorises neighbours; the linear model has four weights and barely notices. Tune *k* on the random split and you would report 0.32 m and deliver 0.64 m. `check_split` reports shared rows, shared runs and the smallest train-test distance against the horizon. A centred rolling mean is a quieter leak (it averages future samples); `CausalRollingMean` uses the past only, and `ExpandingWindowCV` gives Week 7's grid search time-ordered folds.

8. The leak-free workflow

Clean statelessly on the whole log, one run at a time; split by run (or by time, with a gap of at least the horizon); fit the learned stages (impute, clip, scale) on the training rows; transform both sides with that fitted pipeline; fit and evaluate the model and report the split. Result on the class log, nothing changed in the models: mean baseline 3.833 m, linear **0.121 m** (from 2.73 m), k-NN 0.641 m. The linear weights recover the physics, $r(t + 1) \approx r - v \cdot 1\,\mathrm{s}$ plus a small acceleration term; the yaw-rate weight is near zero, which is why median imputation of its frozen segments is acceptable.

Summary. We can now audit a raw log, name the missing-data mechanism and choose its remedy, separate spikes from manoeuvres with local robust rules, keep neighbour-based steps inside one run, scale with training statistics only, and split time-series and grouped data without leakage and prove it. Assignment 2 applies all of this to a fresh log (seed 52), adding a frozen-sensor detector, a learned Tukey clipper and a leak-free `prepare()` function.